WIKIPEDIA 25
25 years of the free encyclopedia

Backpropagation

In machine learning, **backpropagation** is a gradient computation method commonly used for training a neural network in computing parameter updates.

It is an efficient application of the chain rule to neural networks. Backpropagation efficiently computes the gradient of the loss with respect to the network weights for a single input–output example. It does this by propagating derivatives backward, one layer at a time, from the output layer to the input layer, thereby avoiding redundant chain-rule calculations.

Strictly speaking, the term *backpropagation* refers only to an algorithm for efficiently computing the gradient, not how the gradient is used, but the term is often used loosely to refer to the entire learning algorithm. This includes changing model parameters in the negative direction of the gradient, such as by stochastic gradient descent, or as an intermediate step in a more complicated optimizer, such as Adaptive Moment Estimation.[1]

Backpropagation had multiple discoveries and partial discoveries, with a tangled history and terminology (see § History). Some other names for the technique include "reverse mode of automatic differentiation" or "reverse accumulation".[2]

Overview

Backpropagation computes the gradient in weight space of a feedforward neural network, with respect to a loss function. Denote:

- x: input (vector of features)
- y: target output
  For classification, output will be a vector of class probabilities (e.g., $(0.1, 0.7, 0.2)$, and target output is a specific class, encoded by the one-hot/dummy variable (e.g., $(0, 1, 0)$).
- C: loss function or "cost function"[a]
  For classification, this is usually cross-entropy (XC, log loss), while for regression it is usually squared error loss (SEL).
- L: the number of layers
- $W^l = (w^l_{jk})$: the weights between layer $l - 1$ and l, where w^l_{jk} is the weight between the k-th node in layer $l - 1$ and the j-th node in layer l[b]
- f^l: activation functions at layer l
  For classification the last layer is usually the logistic function for binary classification, and softmax (softargmax) for multi-class classification, while for the hidden layers this was traditionally a sigmoid function (logistic function or others) on each node (coordinate), but today is more varied, with rectifier (ramp, ReLU) being common.
- a^l_j: activation of the j-th node in layer l.

In the derivation of backpropagation, other intermediate quantities are used by introducing them as needed below. Bias terms are not treated specially since they correspond to a weight with a fixed input of 1. For backpropagation the specific loss function and activation functions do not matter as long as they and their derivatives can be evaluated efficiently. Traditional activation functions include sigmoid, tanh, and ReLU. Swish,[3] Mish,[4] and many others.

The overall network is a combination of function composition and matrix multiplication:

$$g(x) := f^L(W^L f^{L-1}(W^{L-1} \cdots f^1(W^1 x) \cdots))$$

For a training set there will be a set of input–output pairs, $\{(x_i, y_i)\}$. For each input–output pair (x_i, y_i) in the training set, the loss of the model on that pair is the cost of the difference between the predicted output $g(x_i)$ and the target output y_i:

$$C(y_i, g(x_i))$$

Note the distinction: during model evaluation the weights are fixed while the inputs vary (and the target output may be unknown), and the network ends with the output layer (it does not include the loss function). During model training the input–output pair is fixed while the weights vary, and the network ends with the loss function.

Backpropagation computes the gradient for a *fixed* input–output pair (x_i, y_i), where the weights w^l_{jk} can vary. Each individual component of the gradient, $\partial C / \partial w^l_{jk}$, can be computed by the chain rule; but doing this separately for each weight is inefficient. Backpropagation efficiently computes the gradient by avoiding duplicate calculations and not computing unnecessary intermediate values, by computing the gradient of each layer – specifically the gradient of the weighted *input* of each layer, denoted by δ^l – from back to front.

Informally, the key point is that since the only way a weight in W^l affects the loss is through its effect on the *next* layer, and it does so *linearly*, δ^l are the only data you need to compute the gradients of the weights at layer l, and then the gradients of weights of previous layer can be computed by δ^{l-1} and repeated recursively. This avoids inefficiency in two ways. First, it avoids duplication because when computing the gradient at layer l, it is unnecessary to recompute all derivatives on later layers $l+1, l+2, \ldots$ each time. Second, it avoids unnecessary intermediate calculations, because at each stage it directly computes the gradient of the weights with respect to the ultimate output (the loss), rather than unnecessarily computing the derivatives of the values of hidden layers with respect to changes in weights $\partial a^{l'}_{j'} / \partial w^l_{jk}$.

Backpropagation can be expressed for simple feedforward networks in terms of matrix multiplication, or more generally in terms of the adjoint graph.

Matrix multiplication

For the basic case of a feedforward network, where nodes in each layer are connected only to nodes in the immediate next layer (without skipping any layers), and there is a loss function that computes a scalar loss for the final output, backpropagation can be understood simply by matrix multiplication.[c]

Essentially, backpropagation evaluates the expression for the derivative of the cost function as a product of derivatives between each layer *from right to left* – "backwards" – with the gradient of the weights between each layer being a simple modification of the partial products (the "backwards propagated error").

Given an input–output pair (x, y), the loss is:

$$C(y, f^L(W^L f^{L-1}(W^{L-1} \cdots f^2(W^2 f^1(W^1 x)) \cdots)))$$

To compute this, one starts with the input x and works forward; denote the weighted input of each hidden layer as z^l and the output of hidden layer l as the activation a^l. For backpropagation, the activation a^l as well as the derivatives $(f^l)'$ (evaluated at z^l) must be cached for use during the backwards pass.

The derivative of the loss in terms of the inputs is given by the chain rule; note that each term is a total derivative, evaluated at the value of the network (at each node) on the input x:

$$\frac{dC}{da^L} \cdot \frac{da^L}{dz^L} \cdot \frac{dz^L}{da^{L-1}} \cdot \frac{da^{L-1}}{dz^{L-1}} \cdot \frac{dz^{L-1}}{da^{L-2}} \cdot \ldots \cdot \frac{da^1}{dz^1} \cdot \frac{\partial z^1}{\partial x},$$

where $\frac{da^L}{dz^L}$ is a diagonal matrix.

These terms are: the derivative of the loss function;[d] the derivatives of the activation functions;[e] and the matrices of weights:[f]

$$\frac{dC}{da^L} \circ (f^L)' \cdot W^L \circ (f^{L-1})' \cdot W^{L-1} \circ \cdots \circ (f^1)' \cdot W^1.$$

The gradient ∇ is the transpose of the derivative of the output in terms of the input, so the matrices are transposed and the order of multiplication is reversed, but the entries are the same:

$$\nabla_x C = (W^1)^T \cdot (f^1)' \circ \ldots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C.$$

Backpropagation then consists essentially of evaluating this expression from right to left (equivalently, multiplying the previous expression for the derivative from left to right), computing the gradient at each layer on the way; there is an added step, because the gradient of the weights is not just a subexpression: there's an extra multiplication.

Introducing the auxiliary quantity δ^l for the partial products (multiplying from right to left), interpreted as the "error at level l" and defined as the gradient of the input values at level l:

$$\delta^l := (f^l)' \circ (W^{l+1})^T \cdot (f^{l+1})' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C.$$

Note that δ^l is a vector, of length equal to the number of nodes in level l; each component is interpreted as the "cost attributable to (the value of) that node".

The gradient of the weights in layer l is then:

$$\nabla_{W^l} C = \delta^l (a^{l-1})^T.$$

The factor of a^{l-1} is because the weights W^l between level $l-1$ and l affect level l proportionally to the inputs (activations): the inputs are fixed, the weights vary.

The δ^l can easily be computed recursively, going from right to left, as:

$$\delta^{l-1} := (f^{l-1})' \circ (W^l)^T \cdot \delta^l.$$

The gradients of the weights can thus be computed using a few matrix multiplications for each level; this is backpropagation.

Compared with naively computing forwards (using the δ^l for illustration):

$$\begin{aligned}
\delta^1 &= (f^1)' \circ (W^2)^T \cdot (f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C \\
\delta^2 &= (f^2)' \circ \cdots \circ (W^{L-1})^T \cdot (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C \\
&\vdots \\
\delta^{L-1} &= (f^{L-1})' \circ (W^L)^T \cdot (f^L)' \circ \nabla_{a^L} C \\
\delta^L &= (f^L)' \circ \nabla_{a^L} C,
\end{aligned}$$

There are two key differences with backpropagation:

1. Computing δ^{l-1} in terms of δ^l avoids the obvious duplicate multiplication of layers l and beyond.
2. Multiplying starting from $\nabla_{a^L} C$ – propagating the error *backwards* – means that each step simply multiplies a vector (δ^l) by the matrices of weights $(W^l)^T$ and derivatives of activations $(f^{l-1})'$. By contrast, multiplying forwards, starting from the changes at an earlier layer, means that each multiplication multiplies a *matrix* by a *matrix*. This is much more expensive, and corresponds to tracking every possible path of a change in one layer l forward to changes in the layer $l+2$ (for multiplying W^{l+1} by W^{l+2}, with additional multiplications for the derivatives of the activations), which unnecessarily computes the intermediate quantities of how weight changes affect the values of hidden nodes.

Adjoint graph

For more general graphs, and other advanced variations, backpropagation can be understood in terms of automatic differentiation, where backpropagation is a special case of reverse accumulation (or "reverse mode").[2]

Intuition

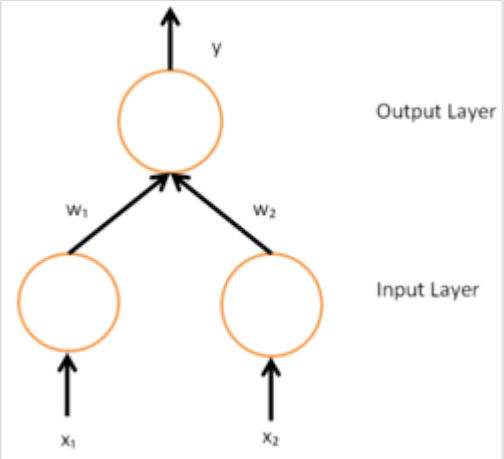
Motivation

The goal of any supervised learning algorithm is to find a function that best maps a set of inputs to their correct output. The motivation for backpropagation is to train a multi-layered neural network such that it can learn the appropriate internal representations to allow it to learn any arbitrary mapping of input to output.[5]

Learning as an optimization problem

To understand the mathematical derivation of the backpropagation algorithm, it helps to first develop some intuition about the relationship between the actual output of a neuron and the correct output for a particular training example. Consider a simple neural network with two input units, one output unit and no hidden units, and in which each neuron uses a linear output (unlike most work on neural networks, in which mapping from inputs to outputs is non-linear)[g] that is the weighted sum of its input.

Initially, before training, the weights will be set randomly. Then the neuron learns from training examples, which in this case consist of a set of tuples (x_1, x_2, t) where x_1 and x_2 are the inputs to the network and t is the correct output (the output the network should produce given those inputs, when it has been trained). The initial network, given x_1 and x_2, will compute an output y that likely differs from t (given random weights). A loss function $L(t, y)$ is used for measuring the discrepancy between the target output t and the computed output y. For regression analysis problems the squared error can be used as a loss function, for classification the categorical cross-entropy can be used.



A simple neural network with two input units (each with a single input) and one output unit (with two inputs)

As an example consider a regression problem using the square error as a loss:

$$L(t, y) = (t - y)^2 = E,$$

where E is the discrepancy or error.

Consider the network on a single training case: $(1, 1, 0)$. Thus, the input x_1 and x_2 are 1 and 1 respectively and the correct output, t is 0. Now if the relation is plotted between the network's output y on the horizontal axis and the error E on the vertical axis, the result is a parabola. The minimum of the parabola corresponds to the output y which minimizes the error E. For a single training case, the minimum also touches the horizontal axis, which means the error will be zero and the network can produce an output y that exactly matches the target output t. Therefore, the problem of mapping inputs to outputs can be reduced to an optimization problem of finding a function that will produce the minimal error.

However, the output of a neuron depends on the weighted sum of all its inputs:

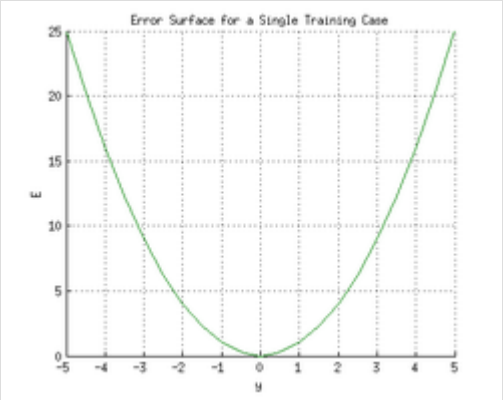
$$y = x_1 w_1 + x_2 w_2,$$

where w_1 and w_2 are the weights on the connection from the input units to the output unit. Therefore, the error also depends on the incoming weights to the neuron, which is ultimately what needs to be changed in the network to enable learning.

In this example, upon injecting the training data $(1, 1, 0)$, the loss function becomes

$$E = (t - y)^2 = y^2 = (x_1 w_1 + x_2 w_2)^2 = (w_1 + w_2)^2.$$

Then, the loss function E takes the form of a parabolic cylinder with its base directed along $w_1 = -w_2$. Since all sets of weights that satisfy $w_1 = -w_2$ minimize the loss function, in this case additional constraints are required to converge to a unique solution. Additional constraints could either be generated by setting specific conditions to the weights, or by injecting additional training data.



Error surface of a linear neuron for a single training case

One commonly used algorithm to find the set of weights that minimizes the error is gradient descent. By backpropagation, the steepest descent direction is calculated of the loss function versus the present synaptic weights. Then, the weights can be modified along the steepest descent direction, and the error is minimized in an efficient way.

Derivation

The gradient descent method involves calculating the derivative of the loss function with respect to the weights of the network. This is normally done using backpropagation. Assuming one output neuron,[h] the squared error function is

$$E = L(t, y)$$

where

L is the loss for the output y and target value t,
t is the target output for a training sample, and
y is the actual output of the output neuron.

In this section, the order of the weight indexes are reversed relative to the prior section: w_{ij} is weight from the ith to the jth unit. [i] For each neuron j, its output o_j is defined as

$$o_j = \varphi(\text{net}_j) = \varphi\left(\sum_{k=1}^{n} w_{kj} x_k\right),$$

where the activation function φ is non-linear and differentiable over the activation region (the ReLU is not differentiable at one point). A historically used activation function is the logistic function:

$$\varphi(z) = \frac{1}{1+e^{-z}}$$

which has a convenient derivative of:

$$\frac{d\varphi}{dz} = \varphi(z)(1-\varphi(z))$$

The input net_j to a neuron is the weighted sum of outputs o_k of previous neurons. If the neuron is in the first layer after the input layer, the o_k of the input layer are simply the inputs x_k to the network. The number of input units to the neuron is n. The variable w_{kj} denotes the weight between neuron k of the previous layer and neuron j of the current layer.

Finding the derivative of the error

Calculating the partial derivative of the error with respect to a weight w_{ij} is done using the chain rule twice:

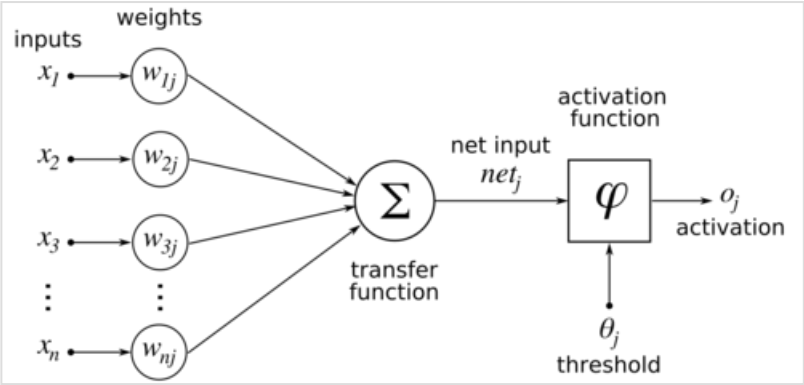


Diagram of an artificial neural network to illustrate the notation used here

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j}\frac{\partial \mathrm{net}_j}{\partial w_{ij}} \qquad \textbf{(Eq. 1)}$$

In the last factor of the right-hand side of the above, only one term in the sum net_j depends on w_{ij}, so that

$$\frac{\partial \mathrm{net}_j}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}}\left(\sum_{k=1}^{n} w_{kj}o_k\right) = \frac{\partial}{\partial w_{ij}} w_{ij}o_i = o_i. \qquad \textbf{(Eq. 2)}$$

If the neuron is in the first layer after the input layer, o_i is just x_i.

The derivative of the output of neuron j with respect to its input is simply the partial derivative of the activation function:

$$\frac{\partial o_j}{\partial \mathrm{net}_j} = \frac{\partial \varphi(\mathrm{net}_j)}{\partial \mathrm{net}_j} \qquad \textbf{(Eq. 3)}$$

which for the logistic activation function

$$\frac{\partial o_j}{\partial \mathrm{net}_j} = \frac{\partial}{\partial \mathrm{net}_j}\varphi(\mathrm{net}_j) = \varphi(\mathrm{net}_j)(1-\varphi(\mathrm{net}_j)) = o_j(1-o_j)$$

This is the reason why backpropagation requires that the activation function be differentiable. (Nevertheless, the ReLU activation function, which is non-differentiable at 0, has become quite popular, e.g. in AlexNet)

The first factor is straightforward to evaluate if the neuron is in the output layer, because then $o_j = y$ and

$$\frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} \qquad \textbf{(Eq. 4)}$$

If half of the square error is used as loss function we can rewrite it as

$$\frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} = \frac{\partial}{\partial y}\frac{1}{2}(t-y)^2 = y - t$$

However, if j is in an arbitrary inner layer of the network, finding the derivative E with respect to o_j is less obvious.

Considering E as a function with the inputs being all neurons $L = \{u, v, \ldots, w\}$ receiving input from neuron j,

$$\frac{\partial E(o_j)}{\partial o_j} = \frac{\partial E(\mathrm{net}_u, \mathrm{net}_v, \ldots, \mathrm{net}_w)}{\partial o_j}$$

and taking the total derivative with respect to o_j, a recursive expression for the derivative is obtained:

$$\frac{\partial E}{\partial o_j} = \sum_{\ell \in L}\left(\frac{\partial E}{\partial \mathrm{net}_\ell}\frac{\partial \mathrm{net}_\ell}{\partial o_j}\right) = \sum_{\ell \in L}\left(\frac{\partial E}{\partial o_\ell}\frac{\partial o_\ell}{\partial \mathrm{net}_\ell}\frac{\partial \mathrm{net}_\ell}{\partial o_j}\right) = \sum_{\ell \in L}\left(\frac{\partial E}{\partial o_\ell}\frac{\partial o_\ell}{\partial \mathrm{net}_\ell}w_{j\ell}\right) \qquad \textbf{(Eq. 5)}$$

Therefore, the derivative with respect to o_j can be calculated if all the derivatives with respect to the outputs o_ℓ of the next layer – the ones closer to the output neuron – are known. [Note, if any of the neurons in set L were not connected to neuron j, they would be independent of w_{ij} and the corresponding partial derivative under the summation would vanish to 0.]

Substituting **Eq. 2**, **Eq. 3** **Eq.4** and **Eq. 5** in **Eq. 1** we obtain:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j}\frac{\partial \mathrm{net}_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j}o_i$$
$$\frac{\partial E}{\partial w_{ij}} = o_i\delta_j$$

with

$$\delta_j = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j} = \begin{cases} \frac{\partial L(t,o_j)}{\partial o_j}\frac{d\varphi(\mathrm{net}_j)}{d\mathrm{net}_j} & \text{if } j \text{ is an output neuron,} \\ \left(\sum_{\ell \in L} w_{j\ell}\delta_\ell\right)\frac{d\varphi(\mathrm{net}_j)}{d\mathrm{net}_j} & \text{if } j \text{ is an inner neuron.} \end{cases}$$

if φ is the logistic function, and the error is the square error:

$$\delta_j = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j} = \begin{cases} (o_j - t_j)o_j(1-o_j) & \text{if } j \text{ is an output neuron,} \\ \left(\sum_{\ell \in L} w_{j\ell}\delta_\ell\right)o_j(1-o_j) & \text{if } j \text{ is an inner neuron.} \end{cases}$$

To update the weight w_{ij} using gradient descent, one must choose a learning rate, $\eta > 0$. The change in weight needs to reflect the impact on E of an increase or decrease in w_{ij}. If $\frac{\partial E}{\partial w_{ij}} > 0$, an increase in w_{ij} increases E; conversely, if $\frac{\partial E}{\partial w_{ij}} < 0$, an increase in w_{ij} decreases E. The new Δw_{ij} is added to the old weight, and the product of the learning rate and the gradient, multiplied by -1 guarantees that w_{ij} changes in a way that always decreases E. In other words, in the equation immediately below, $-\eta\frac{\partial E}{\partial w_{ij}}$ always changes w_{ij} in such a way that E is decreased:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta o_i \delta_j$$

Second-order gradient descent

Using a Hessian matrix of second-order derivatives of the error function, the Levenberg–Marquardt algorithm often converges faster than first-order gradient descent, especially when the topology of the error function is complicated.[6][7] It may also find solutions in smaller node counts for which other methods might not converge.[7] The Hessian can be approximated by the Fisher information matrix.[8]

As an example, consider a simple feedforward network. At the l-th layer, we have

$$x_i^{(l)}, \quad a_i^{(l)} = f(x_i^{(l)}), \quad x_i^{(l+1)} = \sum_j W_{ij} a_j^{(l)}$$

where x are the pre-activations, a are the activations, and W is the weight matrix. Given a loss function L, the first-order backpropagation states that

$$\frac{\partial L}{\partial a_j^{(l)}} = \sum_j W_{ij} \frac{\partial L}{\partial x_i^{(l+1)}}, \quad \frac{\partial L}{\partial x_j^{(l)}} = f'(x_j^{(l)}) \frac{\partial L}{\partial a_j^{(l)}}$$

and the second-order backpropagation states that

$$\frac{\partial^2 L}{\partial a_{j_1}^{(l)} \partial a_{j_2}^{(l)}} = \sum_{j_1 j_2} W_{i_1 j_1} W_{i_2 j_2} \frac{\partial^2 L}{\partial x_{i_1}^{(l+1)} \partial x_{i_2}^{(l+1)}}, \quad \frac{\partial^2 L}{\partial x_{j_1}^{(l)} \partial x_{j_2}^{(l)}} = f'(x_{j_1}^{(l)}) f'(x_{j_2}^{(l)}) \frac{\partial^2 L}{\partial a_{j_1}^{(l)} \partial a_{j_2}^{(l)}} + \delta_{j_1 j_2}$$

where δ is the Dirac delta symbol.

Arbitrary-order derivatives in arbitrary computational graphs can be computed with backpropagation, but with more complex expressions for higher orders.

Loss function

The loss function is a function that maps values of one or more variables onto a real number intuitively representing some "cost" associated with those values. For backpropagation, the loss function calculates the difference between the network output and its expected output, after a training example has

propagated through the network.

Assumptions

The mathematical expression of the loss function must fulfill two conditions in order for it to be possibly used in backpropagation.[9] The first is that it can be written as an average $E = \frac{1}{n}\sum_x E_x$ over error functions E_x, for n individual training examples, x. The reason for this assumption is that the backpropagation algorithm calculates the gradient of the error function for a single training example, which needs to be generalized to the overall error function. The second assumption is that it can be written as a function of the outputs from the neural network.

Example loss function

Let y, y' be vectors in $\mathbb{R}^n$.

Select an error function $E(y, y')$ measuring the difference between two outputs. The standard choice is the square of the Euclidean distance between the vectors y and y':

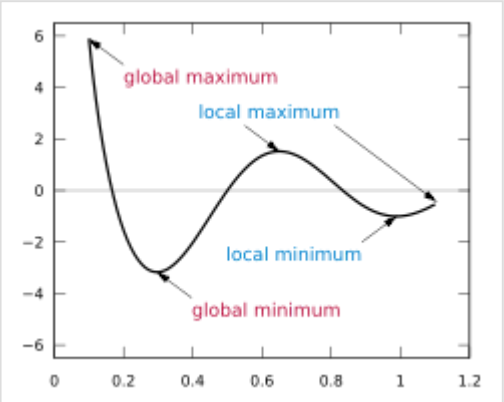
$$E(y, y') = \frac{1}{2}\|y - y'\|^2$$

The error function over n training examples can then be written as an average of losses over individual examples:

$$E = \frac{1}{2n}\sum_x \|(y(x) - y'(x))\|^2$$

Limitations

- Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum; also, it has trouble crossing plateaus in the error function landscape. This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback, but Yann LeCun *et al.* argue that in many practical problems, it is not.[10]
- Backpropagation learning does not require normalization of input vectors; however, normalization could improve performance.[11]
- Backpropagation requires the derivatives of activation functions to be known at network design time.



Gradient descent may find a local minimum instead of the global minimum.

History

Precursors

Backpropagation had been derived repeatedly, as it is essentially an efficient application of the chain rule (first written down by Gottfried Wilhelm Leibniz in 1676)[12][13] to neural networks.

The terminology "back-propagating error correction" was introduced in 1962 by Frank Rosenblatt, but he did not know how to implement this.[14] In any case, he only studied neurons whose outputs were discrete levels, which only had zero derivatives, making backpropagation impossible.

Precursors to backpropagation appeared in optimal control theory since 1950s. Yann LeCun et al credits 1950s work by Pontryagin and others in optimal control theory, especially the adjoint state method, for being a continuous-time version of backpropagation.[15] Hecht-Nielsen[16] credits the Robbins–Monro algorithm (1951)[17] and Arthur Bryson and Yu-Chi Ho's *Applied Optimal Control* (1969) as presages of backpropagation. Other precursors were Henry J. Kelley 1960,[18] and Arthur E. Bryson (1961).[19] In 1962, Stuart Dreyfus published a simpler derivation based only on the chain rule.[20][21][22] In 1973, he adapted parameters of controllers in proportion to error gradients.[23] Unlike modern backpropagation, these precursors used standard Jacobian matrix calculations from one stage to the previous one, neither addressing direct links across several stages nor potential additional efficiency gains due to network sparsity.[24]

The ADALINE (1960) learning algorithm was gradient descent with a squared error loss for a single layer. The first multilayer perceptron (MLP) with more than one layer trained by stochastic gradient descent[17] was published in 1967 by Shun'ichi Amari.[25] The MLP had 5 layers, with 2 learnable layers, and it learned to classify patterns not linearly separable.[24]

Modern backpropagation

Modern backpropagation was first published by Seppo Linnainmaa as "reverse mode of automatic differentiation" (1970)[26] for discrete connected networks of nested differentiable functions.[27][28][29]

In 1982, Paul Werbos applied backpropagation to MLPs in the way that has become standard.[30][31] Werbos described how he developed backpropagation in an interview. In 1971, during his PhD work, he developed backpropagation to mathematicize Freud's "flow of psychic energy". He faced repeated difficulty in publishing the work, only managing in 1981.[32] He also claimed that "the first practical application of back-propagation was for estimating a dynamic model to predict nationalism and social communications in 1974" by him.[33]

Around 1982,[32]:376 David E. Rumelhart independently developed[34]:252 backpropagation and taught the algorithm to others in his research circle. He did not cite previous work as he was unaware of them. He published the algorithm first in a 1985 paper, then in a 1986 *Nature* paper an experimental analysis of the technique.[35] These papers became highly cited, contributed to the popularization of backpropagation, and coincided with the resurging research interest in neural networks during the 1980s.[5][36][37]

In 1985, the method was also described by David Parker.[38][39] Yann LeCun proposed an alternative form of backpropagation for neural networks in his PhD thesis in 1987.[40]

Gradient descent took a considerable amount of time to reach acceptance. Some early objections were: there were no guarantees that gradient descent could reach a global minimum, only local minimum; neurons were "known" by physiologists as making discrete signals (0/1), not continuous ones, and with discrete signals, there is no gradient to take. See the interview with Geoffrey Hinton,[32] who was awarded the 2024 Nobel Prize in Physics for his contributions to the field.[41]

Early successes

Contributing to the acceptance were several applications in training neural networks via backpropagation, sometimes achieving popularity outside the research circles.

In 1987, NETtalk learned to convert English text into pronunciation. Sejnowski tried training it with both backpropagation and Boltzmann machine, but found the backpropagation significantly faster, so he used it for the final NETtalk.[32]:324 The NETtalk program became a popular success, appearing on the *Today* show.[42]

In 1989, Dean A. Pomerleau published ALVINN, a neural network trained to drive autonomously using backpropagation.[43]

The LeNet was published in 1989 to recognize handwritten zip codes.

In 1992, TD-Gammon achieved top human level play in backgammon. It was a reinforcement learning agent with a neural network with two layers, trained by backpropagation.[44]

In 1993, Eric Wan won an international pattern recognition contest through backpropagation.[45][46]

After backpropagation

During the 2000s it fell out of favour, but returned in the 2010s, benefiting from cheap, powerful GPU-based computing systems. This has been especially so in speech recognition, machine vision, natural language processing, and language structure learning research (in which it has been used to explain a variety of phenomena related to first[47] and second language learning.[48])[49]

Error backpropagation has been suggested to explain human brain event-related potential (ERP) components like the N400 and P600.[50]

In 2023, a backpropagation algorithm was implemented on a photonic processor by a team at Stanford University.[51]

See also

- Artificial neural network
- Neural circuit
- Catastrophic interference

- Ensemble learning
- AdaBoost
- Overfitting
- Neural backpropagation
- Backpropagation through time
- Backpropagation through structure
- Three-factor learning

Notes

a. Use C for the loss function to allow L to be used for the number of layers

b. This follows Nielsen (2015), and means (left) multiplication by the matrix W^l corresponds to converting output values of layer $l-1$ to input values of layer l: columns correspond to input coordinates, rows correspond to output coordinates.

c. This section largely follows and summarizes Nielsen (2015).

d. The derivative of the loss function is a covector, since the loss function is a scalar-valued function of several variables.

e. The activation function is applied to each node separately, so the derivative is just the diagonal matrix of the derivative on each node. This is often represented as the Hadamard product with the vector of derivatives, denoted by $(f^l)'\odot$, which is mathematically identical but better matches the internal representation of the derivatives as a vector, rather than a diagonal matrix.

f. Since matrix multiplication is linear, the derivative of multiplying by a matrix is just the matrix: $(Wx)' = W$.

g. One may notice that multi-layer neural networks use non-linear activation functions, so an example with linear neurons seems obscure. However, even though the error surface of multi-layer networks are much more complicated, locally they can be approximated by a paraboloid. Therefore, linear neurons are used for simplicity and easier understanding.

h. There can be multiple output neurons, in which case the error is the squared norm of the difference vector.

i. This order follows (Rumelhart, Hinton & Williams, 1986a):[5] "Δw_{ij} is the change to be made to the weight from the ith to the jth unit"

References

1. Goodfellow, Bengio & Courville 2016, p. 200 (https://www.deeplearningbook.org/contents/mlp.html#pf25), "The term back-propagation is often misunderstood as meaning the whole learning algorithm for multi layer neural networks. Actually, back-propagation refers only to the method for computing the gradient, while another algorithm, such as stochastic gradient descent, is used to perform learning using this gradient."
2. Goodfellow, Bengio & Courville (2016, p. 217 (https://www.deeplearningbook.org/contents/mlp.html#pf36)–218), "The back-propagation algorithm described here is only one approach to automatic differentiation. It is a special case of a broader class of techniques called *reverse mode accumulation*."
3. Ramachandran, Prajit; Zoph, Barret; Le, Quoc V. (2017-10-27). "Searching for Activation Functions". arXiv:1710.05941 (https://arxiv.org/abs/1710.05941) [cs.NE (https://arxiv.org/archive/cs.NE)].

4. Misra, Diganta (2019-08-23). "Mish: A Self Regularized Non-Monotonic Activation Function". arXiv:1908.08681 (https://arxiv.org/abs/1908.08681) [cs.LG (https://arxiv.org/archive/cs.LG)].
5. Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (1986a). "Learning representations by back-propagating errors". *Nature*. **323** (6088): 533–536. Bibcode:1986Natur.323..533R (https://ui.adsabs.harvard.edu/abs/1986Natur.323..533R). doi:10.1038/323533a0 (https://doi.org/10.1038%2F323533a0). S2CID 205001834 (https://api.semanticscholar.org/CorpusID:205001834).
6. Tan, Hong Hui; Lim, King Han (2019). "Review of second-order optimization techniques in artificial neural networks backpropagation" (https://doi.org/10.1088%2F1757-899X%2F495%2F1%2F012003). *IOP Conference Series: Materials Science and Engineering*. **495** (1) 012003. Bibcode:2019MS&E..495a2003T (https://ui.adsabs.harvard.edu/abs/2019MS&E..495a2003T). doi:10.1088/1757-899X/495/1/012003 (https://doi.org/10.1088%2F1757-899X%2F495%2F1%2F012003). S2CID 208124487 (https://api.semanticscholar.org/CorpusID:208124487).
7. Wiliamowski, Bogdan; Yu, Hao (June 2010). "Improved Computation for Levenberg–Marquardt Training" (https://www.eng.auburn.edu/~wilambm/pap/2010/Improved%20Computation%20for%20LM%20Training.pdf) (PDF). *IEEE Transactions on Neural Networks and Learning Systems*. **21** (6): 930. Bibcode:2010ITNN...21..930W (https://ui.adsabs.harvard.edu/abs/2010ITNN...21..930W). doi:10.1109/TNN.2010.2045657 (https://doi.org/10.1109%2FTNN.2010.2045657).
8. Martens, James (August 2020). "New Insights and Perspectives on the Natural Gradient Method". *Journal of Machine Learning Research* (21). arXiv:1412.1193 (https://arxiv.org/abs/1412.1193).
9. Nielsen (2015), "[W]hat assumptions do we need to make about our cost function ... in order that backpropagation can be applied? The first assumption we need is that the cost function can be written as an average ... over cost functions ... for individual training examples ... The second assumption we make about the cost is that it can be written as a function of the outputs from the neural network ..."
10. LeCun, Yann; Bengio, Yoshua; Hinton, Geoffrey (2015). "Deep learning" (https://hal.science/hal-04206682/file/Lecun2015.pdf) (PDF). *Nature*. **521** (7553): 436–444. Bibcode:2015Natur.521..436L (https://ui.adsabs.harvard.edu/abs/2015Natur.521..436L). doi:10.1038/nature14539 (https://doi.org/10.1038%2Fnature14539). PMID 26017442 (https://pubmed.ncbi.nlm.nih.gov/26017442). S2CID 3074096 (https://api.semanticscholar.org/CorpusID:3074096).
11. Buckland, Matt; Collins, Mark (2002). *AI Techniques for Game Programming*. Boston: Premier Press. ISBN 1-931841-08-X.
12. Leibniz, Gottfried Wilhelm Freiherr von (1920). *The Early Mathematical Manuscripts of Leibniz: Translated from the Latin Texts Published by Carl Immanuel Gerhardt with Critical and Historical Notes (Leibniz published the chain rule in a 1676 memoir)* (https://books.google.com/books?id=bOIGAAAAYAAJ&q=leibniz+altered+manuscripts&pg=PA90). Open court publishing Company. ISBN 978-0-598-81846-1.
13. Rodríguez, Omar Hernández; López Fernández, Jorge M. (2010). "A Semiotic Reflection on the Didactics of the Chain Rule" (https://scholarworks.umt.edu/tme/vol7/iss2/10/). *The Mathematics Enthusiast*. **7** (2): 321–332. doi:10.54870/1551-3440.1191 (https://doi.org/10.54870%2F1551-3440.1191). S2CID 29739148 (https://api.semanticscholar.org/CorpusID:29739148). Retrieved 2019-08-04.
14. Rosenblatt, Frank (1962). *Principles of Neurodynamics*. Spartan, New York. pp. 287–298.
15. LeCun, Yann, et al. "A theoretical framework for back-propagation." *Proceedings of the 1988 connectionist models summer school*. Vol. 1. 1988.
16. Hecht-Nielsen, Robert (1990). *Neurocomputing* (http://archive.org/details/neurocomputing0000hech). Internet Archive. Reading, Mass. : Addison-Wesley Pub. Co. pp. 124–125. ISBN 978-0-201-09355-1.

17. Robbins, H.; Monro, S. (1951). "A Stochastic Approximation Method" (https://doi.org/10.1214%2Faoms%2F1177729586). *The Annals of Mathematical Statistics*. **22** (3): 400. doi:10.1214/aoms/1177729586 (https://doi.org/10.1214%2Faoms%2F1177729586).
18. Kelley, Henry J. (1960). "Gradient theory of optimal flight paths". *ARS Journal*. **30** (10): 947–954. doi:10.2514/8.5282 (https://doi.org/10.2514%2F8.5282).
19. Bryson, Arthur E. (1962). "A gradient method for optimizing multi-stage allocation processes". *Proceedings of the Harvard Univ. Symposium on digital computers and their applications, 3–6 April 1961*. Cambridge: Harvard University Press. OCLC 498866871 (https://search.worldcat.org/oclc/498866871).
20. Dreyfus, Stuart (1962). "The numerical solution of variational problems" (https://doi.org/10.1016%2F0022-247x%2862%2990004-5). *Journal of Mathematical Analysis and Applications*. **5** (1): 30–45. doi:10.1016/0022-247x(62)90004-5 (https://doi.org/10.1016%2F0022-247x%2862%2990004-5).
21. Dreyfus, Stuart E. (1990). "Artificial Neural Networks, Back Propagation, and the Kelley-Bryson Gradient Procedure". *Journal of Guidance, Control, and Dynamics*. **13** (5): 926–928. Bibcode:1990JGCD...13..926D (https://ui.adsabs.harvard.edu/abs/1990JGCD...13..926D). doi:10.2514/3.25422 (https://doi.org/10.2514%2F3.25422).
22. Mizutani, Eiji; Dreyfus, Stuart; Nishio, Kenichi (July 2000). "On derivation of MLP backpropagation from the Kelley-Bryson optimal-control gradient formula and its application" (https://coeieor.wpengine.com/wp-content/uploads/2019/03/ijcnn2k.pdf) (PDF). Proceedings of the IEEE International Joint Conference on Neural Networks.
23. Dreyfus, Stuart (1973). "The computational solution of optimal control problems with time lag". *IEEE Transactions on Automatic Control*. **18** (4): 383–385. doi:10.1109/tac.1973.1100330 (https://doi.org/10.1109%2Ftac.1973.1100330).
24. Schmidhuber, Jürgen (2022). "Annotated History of Modern AI and Deep Learning". arXiv:2212.11279 (https://arxiv.org/abs/2212.11279) [cs.NE (https://arxiv.org/archive/cs.NE)].
25. Amari, Shun'ichi (1967). "A theory of adaptive pattern classifier". *IEEE Transactions*. **EC** (16): 279–307.
26. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Masters) (in Finnish). University of Helsinki. pp. 6–7.
27. Linnainmaa, Seppo (1976). "Taylor expansion of the accumulated rounding error". *BIT Numerical Mathematics*. **16** (2): 146–160. doi:10.1007/bf01931367 (https://doi.org/10.1007%2Fbf01931367). S2CID 122357351 (https://api.semanticscholar.org/CorpusID:122357351).
28. Griewank, Andreas (2012). "Who Invented the Reverse Mode of Differentiation?". *Optimization Stories*. Documenta Mathematica, Extra Volume ISMP. pp. 389–400. S2CID 15568746 (https://api.semanticscholar.org/CorpusID:15568746).
29. Griewank, Andreas; Walther, Andrea (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, Second Edition* (https://books.google.com/books?id=xoiiLaRxcbEC). SIAM. ISBN 978-0-89871-776-1.
30. Werbos, Paul (1982). "Applications of advances in nonlinear sensitivity analysis" (http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF). *System modeling and optimization*. Springer. pp. 762–770. Archived (https://web.archive.org/web/20160414055503/http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF) from the original on 14 April 2016. Retrieved 2 July 2017.
31. Werbos, Paul J. (1994). *The Roots of Backpropagation: From Ordered Derivatives to Neural Networks and Political Forecasting*. New York: John Wiley & Sons. ISBN 0-471-59897-6.

32. Anderson, James A.; Rosenfeld, Edward, eds. (2000). *Talking Nets: An Oral History of Neural Networks* (https://direct.mit.edu/books/book/4886/Talking-NetsAn-Oral-History-of-Neural-Networks). The MIT Press. doi:10.7551/mitpress/6626.003.0016 (https://doi.org/10.7551%2Fmitpress%2F6626.003.0016). ISBN 978-0-262-26715-1.
33. P. J. Werbos, "Backpropagation through time: what it does and how to do it," in Proceedings of the IEEE, vol. 78, no. 10, pp. 1550–1560, Oct. 1990, doi:10.1109/5.58337 (https://doi.org/10.1109%2F5.58337)
34. Olazaran Rodriguez, Jose Miguel. *A historical sociology of neural network research (https://web.archive.org/web/20221111165150/https://era.ed.ac.uk/bitstream/handle/1842/20075/Olazaran-RodriguezJM_1991redux.pdf?sequence=1&isAllowed=y)*. PhD Dissertation. University of Edinburgh, 1991.
35. Rumelhart; Hinton; Williams (1986). "Learning representations by back-propagating errors" (http://www.cs.toronto.edu/~hinton/absps/naturebp.pdf) (PDF). *Nature*. **323** (6088): 533–536. Bibcode:1986Natur.323..533R (https://ui.adsabs.harvard.edu/abs/1986Natur.323..533R). doi:10.1038/323533a0 (https://doi.org/10.1038%2F323533a0). S2CID 205001834 (https://api.semanticscholar.org/CorpusID:205001834).
36. Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (1986b). "8. Learning Internal Representations by Error Propagation" (https://archive.org/details/parallel distribu00rume). In Rumelhart, David E.; McClelland, James L. (eds.). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*. Vol. 1 : Foundations. Cambridge: MIT Press. ISBN 0-262-18120-7.
37. Alpaydin, Ethem (2010). *Introduction to Machine Learning* (https://books.google.com/books?id=4j9GAQAAIAAJ). MIT Press. ISBN 978-0-262-01243-0.
38. Parker, D.B. (1985). Learning Logic: Casting the Cortex of the Human Brain in Silicon. Center for Computational Research in Economics and Management Science (Report). Cambridge MA: Massachusetts Institute of Technology. Technical Report TR-47.
39. Hertz, John (1991). *Introduction to the theory of neural computation*. Krogh, Anders., Palmer, Richard G. Redwood City, Calif.: Addison-Wesley. p. 8. ISBN 0-201-50395-6. OCLC 21522159 (https://search.worldcat.org/oclc/21522159).
40. Le Cun, Yann (1987). *Modèles connexionnistes de l'apprentissage* (https://www.sudoc.fr/043586643) (Thèse de doctorat d'état thesis). Paris, France: Université Pierre et Marie Curie.
41. "The Nobel Prize in Physics 2024" (https://www.nobelprize.org/prizes/physics/2024/press-release/). *NobelPrize.org*. Retrieved 2024-10-13.
42. Sejnowski, Terrence J. (2018). *The deep learning revolution*. Cambridge, Massachusetts London, England: The MIT Press. ISBN 978-0-262-03803-4.
43. Pomerleau, Dean A. (1988). "ALVINN: An Autonomous Land Vehicle in a Neural Network" (https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html). *Advances in Neural Information Processing Systems*. **1**. Morgan-Kaufmann.
44. Sutton, Richard S.; Barto, Andrew G. (2018). "11.1 TD-Gammon" (http://www.incompleteideas.net/book/11/node2.html). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.
45. Schmidhuber, Jürgen (2015). "Deep learning in neural networks: An overview". *Neural Networks*. **61**: 85–117. arXiv:1404.7828 (https://arxiv.org/abs/1404.7828). doi:10.1016/j.neunet.2014.09.003 (https://doi.org/10.1016%2Fj.neunet.2014.09.003). PMID 25462637 (https://pubmed.ncbi.nlm.nih.gov/25462637). S2CID 11715509 (https://api.semanticscholar.org/CorpusID:11715509).
46. Wan, Eric A. (1994). "Time Series Prediction by Using a Connectionist Network with Internal Delay Lines". In Weigend, Andreas S.; Gershenfeld, Neil A. (eds.). *Time Series Prediction: Forecasting the Future and Understanding the Past*. Proceedings of the NATO Advanced Research Workshop on Comparative Time Series Analysis. Vol. 15. Reading: Addison-Wesley. pp. 195–217. ISBN 0-201-62601-2. S2CID 12652643 (https://api.semanticscholar.org/CorpusID:12652643).

47. Chang, Franklin; Dell, Gary S.; Bock, Kathryn (2006). "Becoming syntactic". *Psychological Review*. **113** (2): 234–272. doi:10.1037/0033-295x.113.2.234 (https://doi.org/10.1037%2F0033-295x.113.2.234). PMID 16637761 (https://pubmed.ncbi.nlm.nih.gov/16637761).
48. Janciauskas, Marius; Chang, Franklin (2018). "Input and Age-Dependent Variation in Second Language Learning: A Connectionist Account" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6001481). *Cognitive Science*. **42** (Suppl Suppl 2): 519–554. doi:10.1111/cogs.12519 (https://doi.org/10.1111%2Fcogs.12519). PMC 6001481 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6001481). PMID 28744901 (https://pubmed.ncbi.nlm.nih.gov/28744901).
49. "Decoding the Power of Backpropagation: A Deep Dive into Advanced Neural Network Techniques" (https://www.janbasktraining.com/tutorials/backpropagation-in-deep-learning). *janbasktraining.com*. 30 January 2024.
50. Fitz, Hartmut; Chang, Franklin (2019). "Language ERPs reflect learning through prediction error propagation". *Cognitive Psychology*. **111**: 15–52. doi:10.1016/j.cogpsych.2019.03.002 (https://doi.org/10.1016%2Fj.cogpsych.2019.03.002). hdl:21.11116/0000-0003-474D-8 (https://hdl.handle.net/21.11116%2F0000-0003-474D-8). PMID 30921626 (https://pubmed.ncbi.nlm.nih.gov/30921626). S2CID 85501792 (https://api.semanticscholar.org/CorpusID:85501792).
51. "Photonic Chips Curb AI Training's Energy Appetite - IEEE Spectrum" (https://spectrum.ieee.org/backpropagation-optical-ai). *IEEE*. Retrieved 2023-05-25.

Further reading

- Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "6.5 Back-Propagation and Other Differentiation Algorithms" (https://www.deeplearningbook.org/contents/mlp.html#pf25). *Deep Learning* (http://www.deeplearningbook.org). MIT Press. pp. 200–220. ISBN 978-0-262-03561-3.
- Nielsen, Michael A. (2015). "How the backpropagation algorithm works" (http://neuralnetworksanddeeplearning.com/chap2.html). *Neural Networks and Deep Learning* (http://neuralnetworksanddeeplearning.com). Determination Press.
- McCaffrey, James (October 2012). "Neural Network Back-Propagation for Programmers" (https://docs.microsoft.com/en-us/archive/msdn-magazine/2012/october/test-run-neural-network-back-propagation-for-programmers). *MSDN Magazine*.
- Rojas, Raúl (1996). "The Backpropagation Algorithm" (https://page.mi.fu-berlin.de/rojas/neural/chapter/K7.pdf) (PDF). *Neural Networks: A Systematic Introduction*. Berlin: Springer. ISBN 3-540-60505-3.

External links

- Backpropagation neural network tutorial at the Wikiversity
- Bernacki, Mariusz; Włodarczyk, Przemysław (2004). "Principles of training multi-layer neural network using backpropagation" (http://galaxy.agh.edu.pl/~vlsi/AI/backp_t_en/backprop.html).
- Karpathy, Andrej (2016). "Lecture 4: Backpropagation, Neural Networks 1" (https://www.youtube.com/watch?v=i94OvYb6noo&list=PLkt2uSq6rBVctENoVBg1TpCC7OQi31AlC&index=4). *CS231n*. Stanford University. Archived (https://ghostarchive.org/varchive/youtube/20211212/i94OvYb6noo) from the original on 2021-12-12 – via YouTube.
- "What is Backpropagation Really Doing?" (https://www.youtube.com/watch?v=Ilg3gGewQ5U&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=3). *3Blue1Brown*. November

3, 2017. Archived (https://ghostarchive.org/varchive/youtube/20211212/Ilg3gGewQ5U) from the original on 2021-12-12 – via YouTube.

- Putta, Sudeep Raja (2022). "Yet Another Derivation of Backpropagation in Matrix Form" (https://sudeepraja.github.io/BackpropAdjoints/).

Retrieved from "https://en.wikipedia.org/w/index.php?title=Backpropagation&oldid=1351942919"